

全経路点検・自動同期観測システム ―― 構築マニュアル

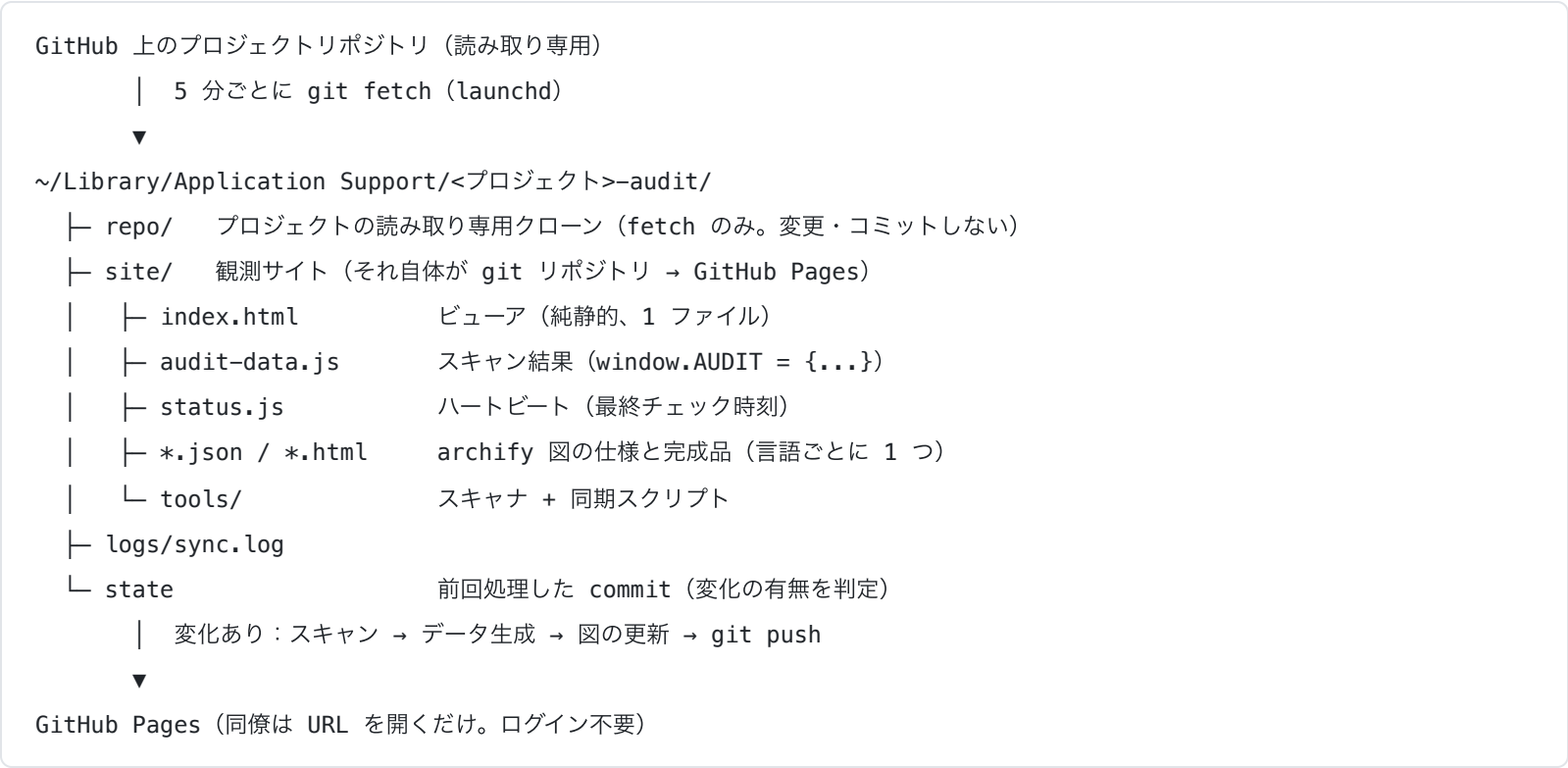
対象：自分のプロジェクトに「Web ページを開くだけで、コード内のボタン / API、どこが切れているか、どこがバグか、どこが未着手かが分かり、しかも **GitHub に合わせて自動更新される**」観測ページを作りたい人。 実例：PANDA FORTUNE 版 → <https://arkai3399-code.github.io/panda-audit/> （このマニュアルがあるリポジトリが、その全ソースです）

0. 完成イメージ

- ・ 上部：システムを選ぶ（1 つのプロジェクトを複数のシステムに分けて表示できる。例：「Web サイト本体 / 社内ツール / 自動化プラットフォーム」＋「システム間の接続」）。右上で言語切替。
- ・ 状態行：「同期済み commit xxxx・最終チェック 10/2 15:51」。2 時間以上チェックされていないと黄色の警告。
- ・ タブ：画面経路（ボタン / フォーム / リンク 1 つずつ）・全体図・API・定時タスク・問題一覧・要確認・今回の変更。
- ・ 左サイドバー：Web ページの実際の順序でグループ化し、件数と最も重いマークを表示。
- ・ 各行に「ソース」リンクがあり、GitHub 上の該当ファイルの該当行へ直接飛べる。
- ・ 全体図：archify で描いたアーキテクチャ図 / データフロー図を「全体図」タブに埋め込む。
- ・ マーク：

マーク	意味
！ 赤	本物のバグ：誤作動する / 結果が違う
▶ 青	完成済み、起動 / 接続を待つだけ
○ 白抜き	未着手：機能がまだ作られていない
◇ 紫	改善 / 要決定
？ 黄	要確認：権限不足で確認できない
「解決済みの可能性・要再確認」	ソース内で元の位置が見つからない / 修正を検出。人の確認待ち

1. 全体構成



- 基本原則：
1. **プロジェクトのリポジトリは読み取り専用**。スキャンは `git archive <commit>` でスナップショットを出して行い、誰の作業ツリーにも触れない。プロジェクトにファイルや workflow を追加しない。
 2. **データとページを分ける**。ビューア `index.html` は `audit-data.js` を読むだけ。別プロジェクトでは主にスキャナを書き換え、ビューアはほぼそのまま。
 3. **自動は自動、人手は人手**。構造的な問題（API に入口なし、部品が未接続）はスキャナが算出する。ロジックのバグや要確認事項は人が登録するが、そのソース位置は「アンカー」で自動的にコードの移動に追従する。
 4. **でっち上げない**。確認できないことは「要確認」に入れ、「完了」とも「問題あり」とも書かない。

2. 用意するもの

もの	用途	備考
常時起動の Mac 1 台	定時同期の実行	電源オフ / スリープ中は同期されず、ページに警告が出る
Node.js 18+	スキャナ、データ生成	
git + プロジェクトリポジトリを読みめるログイン	fetch	端末にある既存の git ログイン（ <code>gh auth login</code> ）を使う。トークンを新たに作ってスクリプトに書き込むことはしない
GitHub アカウント	観測サイトの置き場 + Pages	公開リポジトリなら Pages は無料。非公開リポジトリで Pages を使うには有料プランが必要
archify（Claude Code のスキル）	全体図の作図	任意。無くても動くが、図が無くなる
Claude Code	スキャナの作成、問題の登録	推奨。以下の各ステップはそのまま Claude に任せられる

△ 公開リポジトリにすると、URL を知っている人は誰でも、ファイル名・API パス・問題の説明を見られる。プロジェクトが機密なら非公開リポジトリにするか、社内ネットワークだけに置く。**どんな場合も .env、API Key、token、ユーザーデータを観測サイトに入れないこと。**

3. 作り方

ステップ 1：まず「何を見るか」を決める——システムに分ける

このプロジェクトが実際にどの部分で構成されているかを洗い出し、**分けて表示する。1 枚の図に混ぜない**。例：

- システム A：ユーザーが使う Web サイト（main ブランチ、本番環境）
- システム B：社内ツール（別ブランチ / 別リポジトリ）
- システム C：外部の自動化プラットフォーム（別マシン上のコードパッケージ）
- 接続：A→B、B→C の各接続点がつながっているか

各システムについて、コードの場所（リポジトリ + ブランチ / パッケージ）、デプロイ先、バージョン番号の取得元を明記する。

ステップ 2：スキャナ `tools/analyze.mjs` を書く

入力：`git archive` で出したスナップショットのディレクトリ。出力：`scan.js`。PANDA 版（React + Vercel + Cloud Run）は次の 6 項目を調べている。技術スタックが違う場合は「探し方」の列を置き換える：

調べるもの	PANDA 版の探し方	別プロジェクトでは
① 到達可能性	src/main.jsx から import をたどる。たどり着けないファイル = 「ページ未接続」	エントリファイルを変更。Vue / Next などはそのそれぞれのルーティング / エントリで
② API 一覧	api/**/*.js (Vercel 関数) + サーバ側 server.mjs のルート文字列	自分のルート定義方法に変える (Express app.get、Next app/api、Rails routes…)
③ API の呼び出し元	到達可能なコードで API パスのリテラルを検索。さらに 1 段 : lib 関数内にパスがある → その関数を誰が import しているか	同じ考え方
④ 操作点	JSX の onClick / onSubmit / onChange / <a href> / <Link to>。行番号、近くの文字、ハンドラを記録	自分のテンプレート構文に変える (@click、v-on、addEventListener…)
⑤ グループ分け	「ファイル → 画面領域」の対応表を手で書き、Web ページの実際の順に並べる	人が決める必要あり。一度だけ
⑥ 構造的な問題	呼び出し元のない API = API に入口なし。どこからも import されない部品 = 未接続。プレースホルダのボタン (COMING / 準備中) = 入口未稼働	ルールは共通

Claude Code に任せる場合 : 「tools/analyze.mjs を読んで、同じ出力形式で <プロジェクトのパス> 用のスキャナを書いて。エントリは X、ルート定義は Y」。

「何をするか」はハンドラの**静的な推定** (関数本体に fetch があれば API 呼び出し、setDoc ならデータベース書き込み、navigate なら画面遷移…) なので、ページ上に「実行時トレースではない」と明記する。

ステップ 3 : データ生成 tools/build.mjs → audit-data.js

スキャン結果と人が登録した項目を 1 つのオブジェクトにまとめる :

```

window.AUDIT = {
  title, title_ja, repo: 'https://github.com/<org>/<repo>', checkedAt,
  systems: [{
    id, name, name_ja, short, accent: '#7dd3fc',
    sync: { commit, commitShort, subject, version, date, source },
    where, scope, scale, srcBase: 'github', commit,
    stats: { ops, layers, pages },
    groups: [{ id, title, opCount, layers: [{ file, reachable, ops: [{ line, label, kind, handler, effects, ap
is }] }] },
    apis: [{ path, host, auth, file, line, uiCallers: [{ file, line }], dynamic }],
    crons: { inRepo, needed: [{ what, source }], note },
    issues: [{ kind, level, stale, group, where, what, where_ja, what_ja, source: { file, line, commit, branch
} }],
    verify: [{ item, why, who, stale, source }],
    changes:[{ hash, date, subject, scoring }],
    diagram:{ file: 'overview.html', lead },
  ]},
  links: { lead, diagram, items: [{ from, to, what, status: 'ok'|'bad'|'gap', level, evidence, source } ] },
};

```

多言語：文字列フィールドは x / x_ja のペアで持たせ、ビューアは現在の言語に合わせて取り出す。訳が無ければ中国語にフォールバックする。

人が登録する項目の 3 つのコツ

(a) ソース位置は行番号を直書きせず、正規表現アンカーにする——コードが変わっても行番号が自動で追従する：

```

const anchor = (commit, file, at) => {
  const src = showFile(commit, file);          // git show <commit>:<file>
  if (src == null) return { line: 1, stale: true };          // ファイルが消えた
  if (typeof at === 'number') return { line:at, stale: false };
  const i = src.split('\n').findIndex((l) => at.test(l));
  return i < 0 ? { line: 1, stale: true } : { line: i + 1, stale: false }; // 見つからない → 要再確認
};

M('src/logic/fortuneCalc.js', /^function calcTotal/)    // 使い方

```

アンカーには**その問題コードにしかない行**（関数名、特有の文字列）を選ぶ。} や空行のようなどこにでもある行は選ばない。

(b) 各項目に **level** を書く：bug / ready / todo / note 。判断基準：

- 動かすとエラーになる、数字が違う → bug
- コードは書き終わっていて、入口をつなぐ / スイッチを入れる / 定時実行を設定するだけ → ready
- 機能そのものがまだ書かれていない → todo
- 誤りではないが、決定が必要 / もっと良くできる → note
- 権限が無く確認できない → 問題一覧に入れず「要確認」へ

(c) 「解決済み」を自動判定する：自動でチェックできる項目にはチェック関数を書き、条件を満たしたら `stale` を付ける（ページには「解決済みの可能性・要再確認」と表示）。自動で削除はしない：

```
const RESOLVED = {
  '投稿文 > リンク': () => existsIn(MAIN, 'src/pages/DayPillarCharacterPage.jsx'),
  'DayPillarCharacterCard.jsx': () => grepTree(MAIN, 'src', '^import .*DayPillarCharacterCard').length > 0,
};
```

`grep` には `^` を付けること。付けないと、ファイル自身のコメント内にある `import` 例にマッチしてしまう。

ステップ 4：ビューア `index.html`

このリポジトリの `index.html` をそのままコピーする。依存するのは `audit-data.js` と `status.js` だけ。別プロジェクトで変えるのは次だけ：

- UI 辞書 `UI = {...}`（新しい語がある場合）
- タブ名（例：「定時タスク」が無ければ削除）
- `KIND` 表（問題種別の表示名）

ローカルプレビュー：`python3 -m http.server 8767` → <http://127.0.0.1:8767/>（ダブルクリックで開くと、`file://` の制限で埋め込み図が表示されないことがある）

ステップ 5：全体図（`archify`、任意）

システムごとに 1 枚、さらに「接続」図を 1 枚。言語ごとに仕様を 1 つずつ用意する（`overview.architecture.json` / `overview_ja.architecture.json`）。

```
cd ~/.claude/skills/archify
node bin/archify.mjs deliver architecture <spec.json> <out.html> --quality showcase --repo-root <repo クローン>
--json
```

- 仕様に `sources`（クリックでソースへ飛べる）を含める場合は `--repo-root` が必須。`meta.repository.revision` の `commit` をチェックアウトしていて、`origin` を持つ `git` リポジトリを指定する——読み取り専用クローン `repo/` を使い、開発用の作業ツリーは**使わない**。
- 失敗しても `archify` は前回の `HTML` を残すので、図が壊れることはない。
- 図の中のマーク文字（! ▶ ○ ◇）は、一覧の `level` と一致させる。
- `tools/diagrams_refresh.mjs`：同期のたびに、図の中の**数字**（操作点数、API 数）と**ソース行番号**を最新 `commit` に合わせて更新する。説明文は人が書いたものなので自動では変えない。
- `archify` には日本語の UI 言語が無い：日本語の仕様では `locale` を省略する。図そのものは日本語になり、ビューアのボタンは英語にフォールバックする。

ステップ 6 : GitHub Pages に公開

```
cd site
git init && git add -A && git commit -m "init"
gh repo create <org>/<プロジェクト>-audit --public --source . --push
gh api -X POST repos/<org>/<プロジェクト>-audit/pages -f "source[branch]=main" -f "source[path]=/"
```

- 検索エンジンに載らないよう <meta name="robots" content="noindex, nofollow"> と robots.txt (Disallow: /) を入れる。
- 空ファイル .nojekyll を置く。
- Pages には約 10 分のキャッシュがあるため、push 後に再読み込みしても反映まで少しかかることがある。

ステップ 7 : 自動同期 tools/sync_job.sh + launchd

同期スクリプトの流れ（完全版はこのリポジトリの tools/sync_job.sh、約 50 行）：

```
ロックを取る（2 回同時に走らないように）
プロジェクトリポジトリを git fetch
KEY = 各ブランチの最新 commit + スキャナ / ビューア自身のハッシュ
KEY が state ファイルと同じ → 変化なし：前回のハートビートから 60 分以上なら status.js だけ push して終了
変化あり：
  各ブランチを git archive → 一時ディレクトリ
  analyze → build → audit-data.js
  diagrams_refresh → archify deliver (1 枚ずつ。失敗したら古い図を残して記録)
  status.js を書く (checkedAt / commit / 図の成否)
  観測サイトのリポジトリへ git commit + push
  state を書く
```

KEY に「スキャナ自身のハッシュ」を含めておくと、スキャンのルールやビューアを変えたときも、次回自動で作り直される。

launchd の設定 ~/Library/LaunchAgents/com.<プロジェクト>.audit-sync.plist :

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0"><dict>
  <key>Label</key><string>com.<プロジェクト>.audit-sync</string>
  <key>ProgramArguments</key><array>
    <string>/bin/bash</string>
    <string>/Users/<あなた>/Library/Application Support/<プロジェクト>-audit/site/tools/sync_job.sh</string>
  </array>
  <key>StartInterval</key><integer>300</integer>
  <key>RunAtLoad</key><true/>
  <key>StandardOutPath</key><string>/Users/<あなた>/Library/Application Support/<プロジェクト>-audit/logs/launch
d.out</string>
  <key>StandardErrorPath</key><string>/Users/<あなた>/Library/Application Support/<プロジェクト>-audit/logs/launc
hd.err</string>
</dict></plist>
```

```
launchctl bootstrap gui/$(id -u) ~/Library/LaunchAgents/com.<プロジェクト>.audit-sync.plist    # 有効化
launchctl kickstart -k gui/$(id -u)/com.<プロジェクト>.audit-sync                          # 今すぐ 1 回実行
launchctl bootout  gui/$(id -u)/com.<プロジェクト>.audit-sync                            # 停止
FORCE=1 bash ".../site/tools/sync_job.sh"                                                # 強制的に全部作
り直す
tail -f ".../logs/sync.log"                                                            # ログを見る
```

ビューアは status.js を読み、checkedAt が 120 分を超えていれば「同期が止まっている可能性あり」と表示する。

4. 新しいプロジェクトに移すときのチェックリスト

- ☐ システムに分け、各システムのコードの場所 / デプロイ先 / バージョンの取得元を明記した
- ☐ ~/Library/Application Support/<プロジェクト>-audit/{repo,site,logs} を作り、repo/ にプロジェクトリポジトリをクローンした
- ☐ このリポジトリの index.html と tools/ を site/ にコピーした
- ☐ analyze.mjs を修正した：エントリファイル、ルート / API の定義、操作点の構文、ファイル→グループ表
- ☐ build.mjs を修正した：システム情報、人が登録する項目（アンカー + level）、RESOLVED チェック、接続表
- ☐ ローカルで一度実行し、ブラウザで確認した：数字が合っているか、ソースリンクが正しい行に飛ぶか
- ☐ （任意）archify 図を描き、diagrams_refresh.mjs の「コンポーネント → グループ」対応を修正した
- ☐ GitHub リポジトリ + Pages を作り、noindex を入れた
- ☐ sync_job.sh のパスとランチャ名を修正し、FORCE=1 で手動実行して最後まで通った
- ☐ launchd を入れ、kickstart で 1 回試し、ログに pushed が出た（バックグラウンドからも push できる証拠）
- ☐ ページに「最終チェック」時刻が表示されている

5. よくある落とし穴

症状	原因 / 対処
extra[@]: unbound variable	macOS 標準の bash 3.2 で set -u のとき、空配列を展開するとエラーになる。 \${extra[@]}+"\${extra[@]}" と書く
ログに fatal: path ... does not exist が大量に出る	存在しないファイルを git show すると stderr に出力される。execSync に stdio: ['ignore','pipe','ignore'] を付ける
ある問題が誤って「解決済みの可能性」になる	チェック用の grep がコメント内のサンプルコードにマッチしている。正規表現に ^ を付けるか、より具体的なアンカーにする
図のソースリンクを開くと 404	meta.repository.revision と --repo-root のチェックアウトが一致していない。同期スクリプトでは作図前に git checkout --detach <commit> する
バックグラウンドタスクがデスクトップのフォルダに書き込めない	macOS のプライバシー保護 (TCC)。正本は ~/Library/Application Support/ に置き、デスクトップはミラー扱いにする
ブラウザ自動操作で file:// のページが開けない	python3 -m http.server を起動してからアクセスする
同僚が claude.ai の Artifact リンクを開けない	claude.ai へのログインが必要なため。同僚に見せるものは GitHub Pages に置く
push したのにページが変わらない	Pages のキャッシュ (約 10 分)。またはリポジトリの Actions で pages build が成功しているか確認する

6. できること・できないこと（利用者には正直に伝える）

できること：

- commit に合わせて自動更新：操作点、API、「API に入口なし / 部品が未接続 / プレースホルダの入口」、今回の変更、バージョン番号、図の中の数字と行番号。
- 人が登録した問題もコードの移動に追従する。コードが削除された / 修正済みの場合は再確認を促す。

できないこと：

- ロジックのバグの自動発見**（計算ミス、条件判定の誤り）——人が（または Claude Code にレビューさせて）見つけ、build.mjs に登録する必要がある。
- 本番環境に実際にデプロイされている commit の特定（デプロイ基盤の管理画面の権限が必要）→「要確認」に入れる。
- Mac の電源オフ / スリープ中の同期。24 時間動かす必要があるなら GitHub Actions の定時実行に切り替えられる（読み取り専用トークンが 1 つ必要。リポジトリ管理者が作成して secret に保存する。archify が無いため図は描き直されない）。代替案は tools/ci/README.md を参照。

付録：このリポジトリのファイル対応表

ファイル	役割	別プロジェクトでは
index.html	ビューア	ほぼ変更なし
audit-data.js	生成されたデータ	自動生成
status.js / status.json	ハートビート	自動生成
tools/analyze.mjs	メインシステムのスキャナ	書き直す
tools/analyze_internal.mjs	2 つ目のシステム（ブランチ固有のファイル）のスキャナ	必要に応じて
tools/build.mjs	データの統合 + 人が登録する項目	登録部分を書き直す
tools/diagrams_refresh.mjs	図の中の数字と行番号の更新	対応関係を修正
tools/diagrams_make_ja.mjs	中国語の図仕様 → 日本語の図仕様	辞書を修正
tools/sync_job.sh	定時同期のメインスクリプト	パス / ブランチを修正
tools/ci/	GitHub Actions の代替案	任意
*.architecture.json / *.dataflow.json	archify 図の仕様	描き直す
排査清单.md	このプロジェクトの問題サマリ	書き直す